

Week 2- Algorithm Correctness and Mathematical Induction

Condensed Study Notes

Generated: 2026-05-25 10:15 UTC

Week 2- Algorithm Correctness and Mathematical Induction

This week, we delve into two crucial aspects of algorithms: ensuring they work correctly and a powerful mathematical tool called Mathematical Induction.

Algorithm Correctness refers to proving that an algorithm consistently produces the right output for all valid inputs. It's like making sure a recipe always results in a delicious dish, no matter the slight variations in ingredients or oven temperature.

Mathematical Induction is a proof technique used to establish that a statement is true for all natural numbers (0, 1, 2, 3, ...). It's a bit like a domino effect: if you can show that the first domino falls, and that each falling domino knocks over the next one, then you know all the dominos will eventually fall.

Mathematical Induction typically involves two steps:

1. **Base Case:** Prove that the statement is true for the smallest value (usually 0 or 1).
2. **Inductive Step:** Assume the statement is true for an arbitrary value (let's say 'k') and then prove that it must also be true for the next value ('k+1').

By successfully completing both these steps, we can confidently conclude that the statement holds true for all natural numbers.

Objectives

This section focuses on understanding and verifying the correctness of algorithms.

Key learning goals include:

- **Algorithm Correctness:** Understanding what it means for an algorithm to be correct. Essentially, this means the algorithm produces the right output for every valid input.
- **Verifying Simple Algorithms:** Developing the skills needed to check if simple algorithms work as intended.
- **Loop Invariants:** Learning to use a technique called 'loop invariants' to prove that iterative algorithms (algorithms that repeat a set of steps) are correct.
- **Mathematical Induction:** Applying the proof technique of mathematical induction, which you've already been introduced to, to formally prove the correctness of algorithms.

What is algorithm correctness

Algorithm correctness is all about making sure an algorithm does what it's supposed to do. This means it must produce the expected results for *every single possible input* it might receive.

Think of it like a recipe: correctness means the recipe, when followed exactly, will always produce the intended dish, no matter what brand of flour you use or the exact size of your mixing bowl (within reason, of course!).

However, just getting the right answer isn't the whole story. An algorithm also needs to *finish* its work; it shouldn't run forever without stopping. Because of this, we talk about two main aspects of correctness:

Partial Correctness: *This means that if* the algorithm stops, then it produces the correct result.*

Total Correctness: *This means that the algorithm both stops and* produces the correct result.*

Partial correctness

Partial correctness focuses on whether an algorithm produces the correct output, but it doesn't guarantee that the algorithm will eventually stop running.

Think of it like a recipe: Partial correctness ensures that if you follow the steps and the recipe finishes, you'll get the intended dish. However, it doesn't tell you if the recipe might accidentally ask you to keep stirring a pot forever.

This is important in situations where an algorithm's ongoing operation is more critical than its termination. For example, monitoring systems might run indefinitely to continuously check conditions. For such systems, ensuring that any readings or actions they take are logically accurate (i.e., partially correct) is vital, even though they are designed not to end.

Total correctness

Total correctness means an algorithm not only produces the correct result but also guarantees that it will finish its execution in a finite amount of time.

Think of it like following a recipe. Partial correctness is like knowing that if you follow the steps, you'll end up with a delicious cake. Total correctness adds the guarantee that you will, in fact, finish baking the cake and not be left with a half-baked mess forever.

This is the ideal we aim for in most programming scenarios, as an algorithm that never finishes is often not very useful. It ensures that for any valid input, you'll get the right answer, and you'll get it within a reasonable, predictable time.

Significance of algorithm correctness

Ensuring that algorithms are correct is extremely important, especially in areas where mistakes can have serious consequences.

Think of it like a pilot's flight control system. If the algorithm that manages the aircraft's flight path is incorrect, it could lead to an unsafe or unstable flight. This is a critical example of why correctness matters in aviation software.

Correctness is vital in several key fields:

- **Banking:** For example, a banking algorithm that calculates interest needs to be perfectly accurate. An error here could lead to significant financial discrepancies, meaning people might be charged the wrong amount of interest.
- **Healthcare:** In medical applications, an incorrect algorithm could lead to misdiagnosis or incorrect treatment recommendations.
- **Safety-critical systems:** These are systems where failure could result in injury or death, such as in transportation or industrial control.

In essence, correct algorithms ensure reliability, making software safe and predictable. This means we can trust that the software will behave as expected, every time.

Introduction to Mathematical Induction

Mathematical induction is a powerful proof technique used to demonstrate that a specific statement or formula holds true for all natural numbers.

Think of it like setting up a line of dominoes. To ensure all dominoes fall, you need to do two things:

- **Base Case:** You push over the very first domino. This is like proving the statement is true for the initial natural number, often $n=1$.
- **Inductive Step:** You ensure that if any single domino falls, it will knock over the next one. In induction, this means assuming the statement is true for an arbitrary case (let's say it holds for ' k ') and then proving that it must also hold for the very next case (which would be ' $k+1$ ').

By successfully establishing both the base case and the inductive step, you can confidently conclude that the statement is true for all natural numbers, just like knowing all the dominoes will eventually fall.

Examples of mathematical induction

Mathematical induction is a powerful proof technique. This section demonstrates how to use it with a concrete example.

The goal is to prove a statement for all natural numbers. We'll use the following steps:

1. **Inductive Hypothesis:** Assume the statement is true for an arbitrary natural number, say ' k '. We write this as: Assume it holds for $n=k$.
2. **Inductive Step:** Show that if the statement is true for ' k ', it must also be true for the next natural number, ' $k+1$ '. We write this as: RTP (Required To Prove): Show it holds for $n = k+1$.

Example: Sum of the first n positive integers

Let's prove the formula for the sum of the first n positive integers: $S = (n(n+1))/2$.

- **Inductive Hypothesis:** Assume the formula is true for $n=k$. This means we assume $S = (k(k+1))/2$ is true.

- **Inductive Step:** We need to show that the formula is also true for $n=k+1$. That is, we need to show that the sum of the first $(k+1)$ positive integers is $((k+1)((k+1)+1))/2$, which simplifies to $((k+1)(k+2))/2$.

Strong Induction

Strong induction, also called the second principle of mathematical induction, is another way to prove that a property is true for all natural numbers.

Think of it like this: with simple induction, to prove something for number 'n', you assume it's true for 'n-1'. With strong induction, to prove something for 'n', you assume it's true for *all* numbers less than 'n' (i.e., 0, 1, 2, ..., n-1). This gives you a broader set of assumptions to work with.

How it works

This section details the steps involved in using mathematical induction and strong induction to prove statements.

Mathematical Induction Steps:

- **Base Case:** You first prove that the property you are trying to demonstrate is true for the starting value. This is often the smallest natural number, such as 0 or 1.
- **Inductive Step:** Next, you assume that the property holds true for all values up to a certain number, let's call it 'n'. This assumption is called the **inductive hypothesis**. Based on this assumption, you then prove that the property must also be true for the next number, 'n + 1'.

Think of it like knocking over a line of dominoes. The **base case** is knocking over the first domino. The **inductive step** is proving that *if* one domino falls, it will knock over the next one. If both of these conditions are met, then all the dominoes in the line will eventually fall.

Strong Induction:

Strong induction is a variation of this proof technique that is especially helpful when proving recursive algorithms. In strong induction, for the **inductive step**, you assume the property holds true for *all* natural numbers up to 'n' (i.e., $P(1), P(2), \dots, P(n)$), not just for 'n' itself. You then use this broader assumption to prove the property for 'n + 1'. This is useful because some cases might depend on more than just the immediately preceding case.

Process

To prove an algorithm is correct using mathematical induction, we follow a structured process:

1. **Define the Statement:** First, clearly and precisely state what the algorithm is supposed to achieve. This is the property we want to prove.
2. **Base Case:** We then verify that the algorithm works correctly for the simplest possible input. This is like checking if the first domino in a line will fall.

3. Inductive Step: Next, we show that if the algorithm works correctly for a given input, say $n=k$, it will also work correctly for the next input, $n=k+1$. This is like showing that if one domino falls, it will knock over the next one.

For example, proving the correctness of a recursive function like calculating the factorial of n can be done using this inductive process.

Proving Algorithm Correctness with Induction

This section focuses on how Mathematical Induction, a powerful proof technique, is directly applied to demonstrate that algorithms are correct.

We've previously established that algorithm correctness involves two main aspects:

Partial Correctness: *The algorithm produces the correct output if* it finishes.*

Total Correctness: *The algorithm not only produces the correct output but also guarantees* it will finish in a finite amount of time.*

Mathematical Induction is particularly useful for proving these properties, especially when an algorithm involves loops or processes that repeat a certain number of times, often related to an input size ' n '.

The general approach to using induction for algorithm correctness follows these three key steps:

1. Define the Statement: Clearly state what property you want to prove about the algorithm for any given input size, usually represented by a variable like ' n '. This statement will be the proposition we aim to prove true for all relevant values of ' n '.

2. Establish the Base Case: Prove that the statement holds true for the smallest possible valid input size (often $n=0$ or $n=1$). This is like checking the first domino in a line – if the first one falls, it sets up the possibility for the rest.

3. Demonstrate the Inductive Step: Assume the statement is true for an arbitrary input size ' k ' (this is the **inductive hypothesis**). Then, you must prove that if it's true for ' k ', it must also be true for the next input size, ' $k+1$ '. This is like showing that if any domino falls, it will knock over the *next* one in line.

By successfully completing these steps, we can conclude that the statement holds true for all natural numbers (or all relevant input sizes), thus proving the algorithm's correctness.

Introduction to Loop Invariants

A loop invariant is a condition that is true at the beginning of a loop's execution and remains true after each time the loop runs (each iteration). Think of it like a rule that the loop must always follow.

The main purpose of a loop invariant is to help us prove that iterative algorithms are correct. By showing that this invariant condition holds true, we can demonstrate that the loop is steadily making progress towards its intended goal.

Characteristics of a Good Loop Invariant

A loop invariant is a condition that is true before the loop starts, remains true after each iteration, and helps show that the algorithm has achieved its goal when the loop finishes.

Think of a loop invariant like a safety check during a long journey. You want to ensure that at the start of your trip, certain conditions are met (initialisation). During the journey, after every stop or segment, you want to re-verify those same conditions (maintenance). Finally, when you reach your destination, those conditions should confirm you've arrived correctly (termination).

A well-chosen loop invariant should have these three key properties:

- **Initialisation:** The invariant must be true before the loop begins its first execution.
- **Maintenance:** If the invariant is true at the start of an iteration, it must also be true at the end of that iteration.
- **Termination:** When the loop condition becomes false and the loop ends, the invariant, along with the loop's termination condition, must prove that the algorithm has successfully achieved its intended goal.

Additionally, a good loop invariant should be clear and directly related to what the algorithm is trying to accomplish. Placing the invariant at the beginning of the loop is common practice. A well-selected invariant is valuable for designing, testing, and modifying code, serving as documentation and a basis for proving correctness.

Application of induction to iterative algorithms: Example 1

This section demonstrates how to apply mathematical induction and loop invariants to prove the correctness of an iterative algorithm called POW2(n). The goal is to show that for any natural number n , POW2(n) correctly calculates and returns 2^n .

We will analyze the algorithm by observing how its program variables change with each iteration to find a pattern.

The Statement to Prove: For all natural numbers n , POW2(n) returns 2^n .

Loop Invariant Properties:

We've previously established that a loop invariant must satisfy three properties: initialisation, maintenance, and termination. In this example, we'll focus on how these properties help us prove correctness.

Proof Steps:

1. **Initialisation:** The loop invariant must be true before the first iteration. (Details for this specific algorithm are not provided in the raw content, but this is a crucial step).
2. **Maintenance:** If the loop invariant is true before an iteration, it must also be true after that iteration. This is where we use the 'induction hypothesis' – assuming the invariant holds for some iteration k , we show it holds for iteration $k+1$.

- Let's assume after 't' iterations, a variable 'power' holds the value 2^t (i.e., $\text{power} = 2^t$). This is our inductive hypothesis based on the loop invariant.
- We also observe that another variable 'i' increments with each iteration. If $i_k = k$ after k iterations, then $i_{k+1} = i_k + 1 = k + 1$. This shows 'i' tracks the number of iterations.

3. **Termination:** When the loop finishes, the loop invariant, combined with the loop's termination condition, must imply that the algorithm has produced the correct result.

- The loop continues as long as 'i' is less than 'n'. When 'i' eventually reaches 'n', the loop terminates.
- Since $i_t = t$ (meaning 'i' equals the number of iterations 't'), and the loop terminates when $i_t = n$, we can conclude that the loop terminates after $t = n$ iterations.
- By the loop invariant, at termination (when $t=n$), the 'power' variable will contain 2^t , which is 2^n .

Therefore, POW2(n) terminates with the 'power' variable containing the correct value, 2^n .

Application of induction to iterative algorithms: Example 2

This section demonstrates how to apply mathematical induction to prove the correctness of an iterative algorithm called EVENSUM(n). The goal is to find a loop invariant that holds true throughout the algorithm's execution.

To identify the loop invariant, we examine the program variables within the loop and observe their pattern across different loop iterations.

Let's denote the values of the loop variables at iteration k as i_k and sum_k . We start with initial values:

- $i_0 = 1$
- $\text{sum}_0 = 0$

Through observation of the loop's execution (referencing the EVENSUM(n) algorithm from week 1), we establish the following relationships that hold for any natural number $k \geq 0$:

- $i_k = k + 1$
- $\text{sum}_k = \text{sum from } i=1 \text{ to } k \text{ of } (2i - 2)$

This statement, 'For all natural numbers $k \geq 0$, $i_k = k + 1$ and $\text{sum}_k = \text{sum from } i=1 \text{ to } k \text{ of } (2i - 2)$ ', serves as our loop invariant. We will prove this statement using mathematical induction on k.

Inductive Step:

The proof proceeds by induction on k.

Induction Hypothesis: Assume that for a given $k \geq 0$, the statement holds true: $i_k = k + 1$ and $\text{sum}_k = \text{sum from } i=1 \text{ to } k \text{ of } (2i - 2)$.

EVENSUM(n) Termination and Correctness

This section concludes the proof for the EVENSUM(n) algorithm by demonstrating that it terminates with the correct result.

- **Termination Guarantee:** The loop in EVENSUM(n) is guaranteed to terminate because the loop counter i increments by 1 in each iteration ($i_{k+1} = i_k + 1$). Eventually, i will reach $n + 1$, causing the loop to stop. If we assume this happens after t iterations, the loop will terminate.
- **Connecting Induction to Loop Invariants:** The proof by induction, specifically the initialisation and maintenance phases of the loop invariant, directly supports the correctness of the algorithm. These phases ensure that the invariant holds true throughout the execution of the loop.
- **Final Result:** By the established loop invariant, when the EVENSUM(n) algorithm terminates, the variable it uses will contain the value $\sum_{i=1}^n 2i - 2$. This confirms that the algorithm computes the sum of the first n even numbers correctly.

Inductive Step

The inductive step is where we show that if our assumption (the induction hypothesis) is true for a certain value, it must also be true for the next value.

Induction Hypothesis:

Assume that for POW2(k), where k is greater than or equal to 2, the function correctly returns 2^k for all input values from 0 up to k (i.e., for 0, 1, 2, ..., k).

Think of it like a chain reaction. If you know the first domino falls (base case), and you can show that if any domino falls, the *next* one will also fall (inductive step), then you know all the dominos will eventually fall.

Strong Induction: Recursive POW2 Example

This section begins applying strong induction to prove the correctness of recursive algorithms, using the POW2(n) function as an example.

Statement to Prove: For all $n \geq 0$, the recursive function POW2(n) correctly returns 2^n .

Base Case ($n=0$):

- When n is 0, POW2(0) returns 1. This matches the claim, as $2^0 = 1$. The base case holds true.

Strong Induction for Recursive EVENSUM

This section demonstrates the application of strong induction to prove the correctness of the recursive EVENSUM(n) algorithm. The goal is to show that EVENSUM(n) correctly computes the sum of the first n even numbers, which is represented as $\sum_{i=1}^n 2i - 2$.

Claim: For all $n \geq 1$, EVENSUM(n) returns $\sum_{i=1}^n 2i - 2$.

Base Cases:

- For $n = 1$, EVENSUM(1) returns 0, which matches the claim ($\sum_{i=1}^1 2i - 2 = 2(1) - 2 = 0$).
- For $n = 2$, EVENSUM(2) returns 2, which also matches the claim ($\sum_{i=1}^2 2i - 2 = (2(1) - 2) + (2(2) - 2) = 0 + 2 = 2$).

Inductive Step:

- **Induction Hypothesis:** Assume that for some integer $k \geq 2$, $\text{EVENSUM}(m)$ correctly returns $\sum_{i=1}^m 2i - 2$ for all integers m such that $0 \leq m \leq k$.

- **To Prove:** We need to show that $\text{EVENSUM}(k+1)$ also returns the correct sum, which is $\sum_{i=1}^{k+1} 2i - 2$.

- **Analysis of $\text{EVENSUM}(k+1)$:** The recursive definition of $\text{EVENSUM}(k+1)$ is used here. Based on the induction hypothesis, we know that $\text{EVENSUM}(k)$ correctly computes $\sum_{i=1}^k 2i - 2$. The structure of the recursive call for $\text{EVENSUM}(k+1)$ allows us to relate its output to the known correct output of $\text{EVENSUM}(k)$.

Conclusion: By successfully establishing the base cases and the inductive step, we conclude that for all $n \geq 1$, $\text{EVENSUM}(n)$ correctly returns $\sum_{i=1}^n 2i - 2$.

Activities

This section provides opportunities to practice the concepts of algorithm correctness and mathematical induction.

Activity 1: Practice proving correctness using induction.

- **Example Problem:** Prove the correctness of a recursive algorithm, such as a function that calculates the n^{th} factorial.

Activity 2: Identify loop invariants in simple iterative algorithms.

- **Example Problem:** Verify the loop invariant for an algorithm designed to find the maximum element within an array.

Summary

This section serves as a recap of the key concepts covered. We have learned about two powerful tools for proving algorithm correctness: Mathematical Induction and Loop Invariants.

We also defined what algorithm correctness means – ensuring an algorithm not only produces the right output but also finishes its execution. The importance of this was discussed, highlighting why reliable algorithms are crucial.

Finally, we put these tools into practice by applying them to simple algorithms, reinforcing our understanding through hands-on exercises.

Further tasks

To solidify your understanding of proving algorithm correctness, it's beneficial to engage in further practice.

- **Practice Problems:** Actively seek out and work through more problems related to proving algorithm correctness using mathematical induction and loop invariants. You can find these in personal research or various online resources.

- **Real-World Preparation:** Take some time to think about other real-life scenarios where ensuring the correctness of a process or system is absolutely essential. This helps to reinforce why these concepts are so important.